

Neuron Map

Тест-план (Test Plan)

Нативное приложение macOS для интеллект-карт и изучения материала

Версия документа	1.0
Дата	25.06.2026
Связанные документы	Neuron Map — SRS v2.0; Neuron Map — Test Cases; Neuron Map — Bug Reports
Тестируемая версия	v0.9.1 (по состоянию на итерацию 11 разработки)

1. Введение

Данный тест-план описывает подход, объём, ресурсы и график тестирования приложения Neuron Map — нативного macOS-приложения для создания интеллект-карт с поддержкой вложенных подтем. План основан на требованиях документа SRS v2.0 и учитывает фактическую историю разработки — 11 итераций обратной связи между тестировщиком и разработчиком.

2. Цели тестирования

- Подтвердить соответствие реализованного функционала требованиям SRS (FR-1...FR-20, NFR-1...NFR-8).
- Выявить дефекты функциональности, взаимодействия жестов, локализации и производительности на раннем этапе.
- Обеспечить отсутствие регрессий при последующих изменениях кода за счёт повторного прогона тест-кейсов, связанных с ранее исправленными дефектами.
- Подтвердить готовность сборки к передаче внутри команды (внутренняя дистрибуция без Mac App Store).

3. Объект тестирования

Приложение Neuron Map, собираемое локально через Xcode + XcodeGen из исходного кода проекта, версия сборки v0.9.1. Тестирование выполняется на реальном устройстве Mac (Apple Silicon) в конфигурации Debug, запускаемой через Xcode Run либо экспортированный .app.

4. Объём тестирования

4.1. В объёме (In Scope)

- Создание, редактирование, удаление нейронов, включая диалог подтверждения удаления.
- Вложенные карты (рекурсивная навигация по подтемам).
- Связи между нейронами: создание, перемещение, автоматическое удаление.
- Масштабирование и панорамирование холста, включая крайние значения zoom.
- Навигация: breadcrumbs, история назад/вперёд, поиск.
- Несколько независимых карт (boards): создание, переключение, переименование, удаление.
- Локализация интерфейса (RU/EN), включая диалоги и справку по горячим клавишам.
- Персистентность данных (сохранение между перезапусками).
- Производительность интерфейса при повышенной нагрузке (20+ нейронов).
- Сборка проекта без ошибок компиляции.

4.2. Вне объёма (Out of Scope)

- Совместная работа нескольких пользователей и облачная синхронизация (не реализованы в текущей версии).
- Публикация и тестирование через Mac App Store (используется прямая внутренняя дистрибуция).
- Автоматизированное UI-тестирование (в текущем цикле тестирование выполняется вручную; см. раздел б).
- Тестирование на Intel-based Mac (целевая конфигурация — Apple Silicon).

5. Стратегия тестирования

5.1. Виды тестирования

Вид	Описание	Когда применяется
Функциональное	Проверка соответствия каждого FR ожидаемому поведению	После реализации каждой фичи
Регрессионное	Повторный прогон кейсов, связанных с ранее найденными багами	Перед каждой новой сборкой
Исследовательское (exploratory)	Свободное исследование новых экранов/фич без формальных кейсов	После каждой итерации, до формализации кейсов
UI/UX-тестирование	Визуальное выравнивание, соответствие стилю приложения, а не веб-виджетам	При изменениях интерфейса
Тестирование производительности	Проверка FPS и отзывчивости при 20+ нейронах	Перед релизными сборками
Локализационное	Полнота и корректность переводов RU/EN, отсутствие «сырых» ключей	После изменений в строковом каталоге
Смоук-тестирование сборки	Проверка успешной компиляции и запуска после генерации проекта	После каждого `xcodegen generate` и перед тест-циклом

5.2. Подход

Тестирование выполняется вручную тестировщиком на реальном устройстве, так как приложение нативное и не может быть собрано или запущено в изолированном окружении ассистента-разработчика. Каждая итерация включает: получение обновлённой сборки → прогон релевантных тест-кейсов (новый функционал + регрессия) → фиксация дефектов в формате Bug Report → передача разработчику → повторная проверка после исправления.

6. Тестовое окружение

Параметр	Значение
ОС	macOS 13+ (целевая версия), тестирование на актуальной версии macOS
Архитектура	Apple Silicon
Способ запуска	Xcode Run (Debug) и экспортированный .app (Product → Show Build Folder / Archive)
Инструменты сборки	Xcode, XcodeGen (`project.yml` → `.xcodeproj`), Homebrew
Подпись приложения	Бесплатный Apple ID (без платной Apple Developer подписки); снятие карантина Gatekeeper вручную
Хранилище данных	SwiftData, локально на диске
Языки интерфейса	Русский (основной), английский

7. Критерии входа (Entry Criteria)

- Получена сборка, успешно компилирующаяся в Xcode без ошибок (см. NFR-8).
- Документ SRS согласован и не содержит открытых вопросов по тестируемому функционалу.
- Тестовое окружение подготовлено: устройство Apple Silicon, актуальный Xcode, XcodeGen установлен.

8. Критерии выхода (Exit Criteria)

- Все тест-кейсы с приоритетом «Высокий» выполнены с результатом Passed.
- Отсутствуют открытые дефекты severity «Блокирующий»/«Критический», включая ошибки сборки.
- Количество открытых дефектов severity «Значительный» — не более 2.
- Все дефекты, ранее закрытые как «Исправлен», повторно подтверждены соответствующим регрессионным тест-кейсом.
- Известные ограничения (например, текущий открытый дефект BUG-14) задокументированы и согласованы как принятый риск для данной версии.

9. Тестовые поставляемые результаты (Test Deliverables)

- SRS
- Test Plan
- Test Cases
- Bug Reports
- Traceability Matrix
- Test Summary Report

10. График итераций тестирования

Тестирование велось циклично вслед за итерациями разработки. Ниже — сопоставление ключевых итераций с фокусом тестирования на каждой из них.

Итерация	Версия	Фокус тестирования	Найдено дефектов
1–2	v0.1	Первичная сборка окружения, базовая работоспособность (создание карты, узлов)	0 (проблемы с окружением сборки, не баги ПО)
3	v0.2	Навигация, статус-панель, связи, удаление	0 (получена обратная связь по UX)
4	v0.3	Zoom/pan, редактирование, поиск, Review Mode, иконка приложения	0 (функционал добавлен по запросу)
5	—	Исследование конкурентов, вес хранилища (сборка не поставлена)	—
6	v0.5	Наложение новых нейронов, потеря ориентира при zoom, дрожание связей, hotkeys	BUG-01, BUG-02, BUG-03 (частично), BUG-04
7	v0.6	Повторная проверка zoom и связей, цвет фона карты	BUG-02 (повторно), BUG-03 (закрыт)
8	v0.7	Фриз при zoom-out, «мастер»-перетаскивание, мультикарты, стиль индикатора Link	BUG-05, BUG-06
9	v0.8	Кривой сайдбар, неотзывчивая корзина, FPS, конфликт жестов	BUG-07, BUG-08, BUG-09

Итерация	Версия	Фокус тестирования	Найдено дефектов
10	v0.9	Навигация поиска, hover-баг сайдбара, подтверждение удаления, FPS zoom-out	BUG-10, BUG-11, BUG-12, BUG-13
11	v0.9.1	Смоук-тест сборки после правок BreadcrumbView	BUG-14 (открыт)

11. Роли и ответственность

Роль	Ответственность
Тестировщик (Захар)	Ручное тестирование сборок, оформление дефектов, регрессия, приёмка по критериям выхода
Разработчик	Реализация функционала, диагностика первопричины и исправление дефектов, самопроверка перед передачей сборки

12. Риски и меры по их снижению

Риск	Влияние	Мера снижения
Приложение нативное — не может быть протестировано автоматически в среде ассистента-разработчика	Средний — увеличивает цикл обратной связи	Каждая сборка проверяется тестировщиком вручную на реальном устройстве; разработчик проводит структурный самоанализ (баланс скобок, поиск висячих ссылок) перед передачей
Отсутствие платной Apple Developer подписки	Низкий — влияет только на способ распространения	Используется бесплатный Apple ID и ручное снятие карантина Gatekeeper; задокументировано в README
Повторное появление визуально схожих, но технически разных багов (например, дрожание связей)	Средний — риск ложного ощущения «уже исправлено»	Регрессионные тест-кейсы явно помечены как связанные с конкретным BUG-ID и прогоняются при каждой новой сборке
Незакрывающаяся ошибка сборки блокирует дальнейшее тестирование новых фич	Высокий	BUG-14 зафиксирован с приоритетом «Высочайший»; до его закрытия дальнейшее функциональное тестирование новых фич приостановлено

13. Управление дефектами

Жизненный цикл дефекта: Обнаружен → Зафиксирован в Bug Report → Передан разработчику → Исправлен → Повторно проверен тестировщиком (регрессия) → Закрыт. При повторном обнаружении дефекта после исправления статус переводится в «Исправлен повторно» с указанием обеих итераций в истории версии.

Приоритизация дефектов при работе в очереди на исправление осуществляется по severity: Блокирующий/Критический → Значительный → Незначительный, при равном severity — по влиянию на основной пользовательский сценарий (создание → редактирование → сохранение).